

idris-mcp

An MCP server and client implemented from scratch in Idris2, using real container theory for compile-time protocol safety

Robert Iannuzzo · project report · built on André Videla's container-compendium (Ghani/Abbott/Altenkirch containers)

1. What this is

MCP (Model Context Protocol) is the JSON-RPC-based protocol that connects LLM hosts (like Claude Code) to external tools. There is no existing Idris2 SDK for it, so this project implements the protocol itself — JSON-RPC 2.0 framing, a stdio transport, and message handling — rather than wrapping an existing library.

The point isn't “MCP in one more language.” It's to use Idris2's dependent types to make protocol conformance a compile-time property. The set of methods a server exposes is modeled as a real *container* in the Abbott/Altenkirch/Ghani sense — a type of shapes paired with a response type indexed by shape — built on an actual, independently-authored formalization of container theory rather than a hand-rolled stand-in.

Everything described below has been verified running: the server and client talk real JSON-RPC over real stdio pipes; the server is registered with Claude Code and was called live by the model in a real conversation; and a deliberately mismatched handler was shown to fail to compile, in the actual server code, with a precise type error.

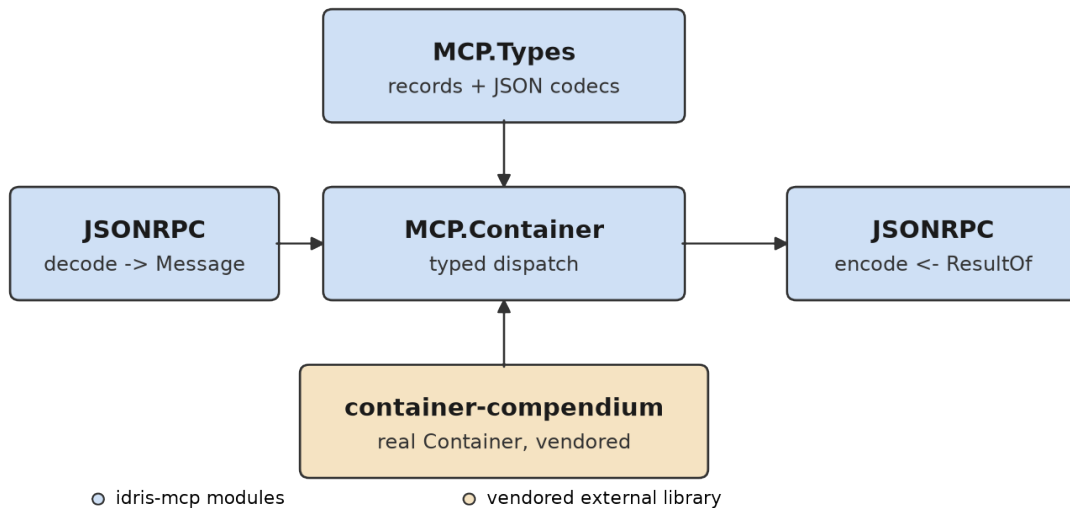
2. Architecture

2.1 File layout

Path	Role
src/JSONRPC.idr	JSON-RPC 2.0 envelope: request/notification/response/error records, encode/decode. Includes a fix for a real Language.JSON bug (integer ids serializing as floats).
src/MCP/Transport.idr	The stdio transport: newline-delimited JSON on stdout, a read loop over stdin, logging routed to stderr so it never corrupts the protocol stream.
src/MCP/Types.idr	Wire-format records for initialize/tools-list/tools-call results, and their JSON encoders. The “knows how to serialize a value” layer.
src/MCP/Container.idr	The typed core: Method, ResultOf, MCP : Container, and the exhaustive dispatch / encodeResult / decodeRequest functions. Described in detail in Section 3.
src/Server.idr	Glue only: reads a line, decodes it, calls into MCP.Container, encodes the result, writes it back. No typed logic of its own.
src/Client.idr	A separate, deliberately untyped Phase-0 client: spawns a server subprocess, drives the handshake/list/call sequence with plain JSON. Does not go through MCP.Container.

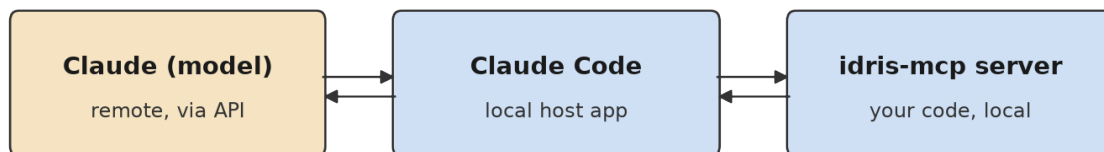
Path	Role
vendor/container-compendium/	The vendored, patched slice of André Videla's real container-compendium library that MCP.Container depends on. See Section 4.
vendor/.../mcp-demo/McpReal.idr	A standalone (no transport) demo showing the client modeled as a literal value of the library's dependent-lens type, (<code>=%></code>). Not wired into the live transport.

2.2 Request path through the live server



Server.idr wires this together in order: a raw line arrives on stdin, JSONRPC decodes the envelope, MCP.Container.decodeRequest turns it into a typed Method value (or rejects it), dispatch produces a value of exactly that method's ResultOf type, encodeResult turns it back into JSON, and JSONRPC writes the response line to stdout. MCP.Types supplies the underlying record types and JSON codecs; container-compendium supplies the actual Container type and its `!>` constructor that MCP.Container is built on.

2.3 Who's actually running what, in the live demo



Claude (the model) runs remotely and never touches the server binary directly — it only ever sees the abstract tool description (name, description, JSON Schema) and later the text result. Claude Code is the local host: it spawns idris-mcp's server as a child process and speaks JSON-RPC to it over a stdio pipe, the same mechanism this project's own Client.idr uses. The server itself has no network dependency and no awareness of Anthropic at all.

2.4 Build system

server.ipkg, client.ipkg, and container-compendium.ipkg are all built with plain `idris2 --build / --install`. The Idris2 package manager, `pack`, was bootstrapped during development (it produced the specific nightly Idris2 build used throughout) but nothing in the running project actually depends on it — a two-executable project with one vendored local dependency doesn't need it.

2.5 What's connected right now

- **idris-mcp/build/exec/server** and **.../client** — rebuild and run standalone; verified end to end (handshake, tools/list, tools/call, clean exit).
- **idris-mcp/vendor/container-compendium/mcp-demo/build/exec/mcp-demo** — the in-process client-as-lens demo against the real library.
- The server is registered with Claude Code (`c laude mcp list` shows Connected) and has been called live by the model from a real chat session.

3. Containers and type theory

3.1 A container is a request/response interaction structure

In the Abbott/Altenkirch/Ghani sense, a container is a pair (S, P) of a type of shapes S and a response family $P : S \rightarrow \text{Type}$ indexed by shape. Its extension, the functor sending X to $\Sigma(s:S). P(s) \rightarrow X$, is the classic way to represent strictly-positive datatypes as polynomial functors, and containers themselves model *interaction structures*: a system you can query with something in S , which answers with something in $P(s)$ — for that specific s .

`MCP.Container.idr`'s `MCP : Container = (m : Method) !> ResultOf m` is a direct instance of this, not a metaphor. `Method` is the shape type (`initialize / tools-list / tools-call`, each carrying whatever data that variant needs); `ResultOf` is the response family, defined by pattern-matching on `Method` to the exact record type each variant's response must have.

3.2 The detection mechanism: dependent elaboration, not “containers” per se

The compile-time safety comes from ordinary dependent type theory, not specifically from the `Container` abstraction. `dispatch : (m : Method) -> ResultOf m` is a dependent function type (a Π -type) over `Method`'s response family. When Idris elaborates one clause, say `dispatch ListTools = ...`, it substitutes the pattern into the codomain and actually *computes* `ResultOf ListTools` down to a concrete type, `List Tool`, using `ResultOf`'s own definition. The right-hand side's inferred type is then unified against that computed type. A swapped or mismatched clause fails unification and is rejected with a precise error at that exact line — verified directly by deliberately breaking a clause and rebuilding.

This is standard dependent-type-checking machinery (the same mechanism any indexed type family uses in Agda, Coq, or Lean), not something bespoke to container theory. What the container framing contributes is the *meaning* of `Method` and `ResultOf` as a request/response interaction structure, and access to the wider algebra built on top of that structure (below).

3.3 Two notions of container morphism, and why MCP needs the lens

The compendium formalizes (at least) two distinct notions of morphism between containers: `Chart` — covariant on both shapes and positions — and `(= %>)`, a dependent lens: shapes map forward, but positions map *contravariantly*, `bwd : (x : c1.request) -> c2.response (fwd x) -> c1.response x`. These are not interchangeable. An MCP client needs the lens: it commits to a request, then must interpret whatever response comes back *for that specific request* — exactly what the contravariant `bwd` captures.

This is the same abstract shape used in the optics/open-games literature (Jules Hedges' work, among others) and in backprop-as-a-lens (forward pass predicts, backward pass consumes a gradient). In `vendor/.../mcp-demo/McpReal.idr`, the MCP client is modeled as a literal value of this real `(= %>)` type: a Moore machine mapping state $\rightarrow$ next request and typed response $\rightarrow$ next state, with identity and composition (`| %>`) already proven to form a category via `Reflexive/Transitive/Preorder` instances in the library. This design is verified against the real library but is not yet wired into the live stdio transport — `Client.idr` is still the plain, untyped Phase-0 client.

3.4 Getting the real library to compile: what had to be patched

The container-compendium repository is a Quartz-powered digital-garden site, not a normal Idris2 package. Most of its foundational modules exist only as literate `.idr.md` notes, with real source hidden inside HTML comments and code fences for web rendering. An extraction script (`tools/extract_literate_idris.py`) recovers the compilable source. Two genuine defects were then found and patched, in the vendored copy only (the original site repository was left untouched):

- **Unicode operators.** No Idris2 build checked — stable 0.8.0 or the pack-pinned nightly — accepts non-ASCII operator characters (relational composition, ring-operator composition, the tensor symbol $\otimes$, sequencing triangles, etc.); the lexer's `isOpChar` is a fixed ASCII whitelist. Unused Unicode fixity declarations were commented out in the vendored `Data/Category/Ops.idr` and `Data/Ops.idr`.
- **A nonexistent `autobind` keyword** inside a record body in `Extension/Definition.idr`, present in neither Idris2 build checked. Removed as vestigial — ordinary record field dependency already covers what it seemed to be for.

The vendored tree (`vendor/container-compendium/src/`) contains the entire extracted compendium, all 208 modules — but only 6 are declared in `container-compendium.ipkg` and actually built/installed: `Data.Boundary`, `Data.Category.Ops`, `Data.Container.Definition`, `Data.Container.Extension.Definition`, `Data.Container.Morphism.Chart`, and `Data.Container.Morphism.Definition`. The rest remain available, unbuilt, for later phases.

3.5 Scope, precisely: per-method typing vs. per-tool typing

Type safety today is at the *method* level: `initialize`, `tools/list`, and `tools/call` each get their own checked `ResultOf`. It is not yet at the *tool* level — a `CallTool "hello" args` and a hypothetical `CallTool "add" args` are the same shape as far as the type system is concerned; both carry an untyped `Maybe JSON` blob of arguments and both produce the same `CallToolResult` type. Mixing up two tools' handlers, or accepting a malformed-for-that-tool argument, is currently only caught at runtime. The natural next step is to model “the set of tools + their argument shapes” as its own container, and use the library's already-vendored (but currently unused) `Ex extension` type to type each tool's call individually, the same way `Method/ResultOf` types each method individually today.

Honesty note: the compendium's structure for composing containers — products, coproducts, tensor, sequential composition — is real and formalized in the library, and is architecturally consistent with everything built here, but is not yet exercised. This project currently uses exactly one flat container (MCP) and one morphism type ($=\%>$); the “composes for free” claims of container theory are available, not yet demonstrated.

4. Verified demo arc

- **Client ↔ server, real transport.** `./build/exec/client` spawning `./build/exec/server`: real JSON-RPC 2.0 over real stdio pipes, full handshake / tools-list / tools-call sequence, clean exit.
- **Real host.** The server registered and connected in Claude Code (`claude mcp list` → Connected); `/mcp` showed the hello tool with its correct name, description, and JSON Schema; the model then called it live in a normal conversation and the response was returned end to end.
- **Compile-time rejection.** Deliberately swapping the `ListTools` branch's return value for a `CallToolResult` produced a precise compiler error — Mismatch between: `CallToolResult` and `List Tool` — in the live server code, not a toy example, confirmed by a clean rebuild after reverting.
- **Container/lens design, real library.** `vendor/.../mcp-demo/McpReal.idr` runs against the real, patched container-compendium and produces output identical to the earlier hand-rolled prototype, with the same negative-demo rejection reproduced against the real ($=\%>$) type.

5. Natural next steps

- Per-tool argument/result typing via the library's `Ex` (extension) type, replacing the current untyped `Maybe JSON` argument blob.
- Wire the client-as-lens design from `McpReal.idr` into the live `Client.idr` / stdio transport.
- Exercise container composition (coproduct/tensor) to combine `tools/*`, `resources/*`, and `prompts/*` as separate composed interfaces, rather than one flat `Method` type.